DOCX output writer — deep multi-angle research (§11.4.150)

Revision: 1 **Last modified:** 2026-06-15T00:00:00Z **Scope:** unified-translator DOCX (`.docx`) OUTPUT support — `pkg/ebook/docx_writer.go` + `cmd/unified-translator/main.go` output-format switch. **Authority:** constitution §11.4.150 (deep-research-first), §11.4.99 (latest-source), §11.4.6 (no-guessing), §11.4.3 (honest typed error on unavailable dependency).

Gap (FACT)

`unified-translator` supports OUTPUT formats `epub` / `fb2` / `html` / `htm` / `txt` / `md` only (`cmd/unified-translator/main.go` `generateOutput`, default case returns `unsupported output format %q (supported: .epub, .fb2, .html, .txt, .md)`). DOCX is INPUT-only in `pkg/ebook` (`docx_parser.go`; there is no `docx_writer.go`). DOCX is a genuinely useful manuscript-review OUTPUT the operator video-tested.

Options considered

(a) Pandoc-backed (render content → markdown → docx via `pandoc`)

- Pandoc 3.9.0.2 is installed (`/opt/homebrew/bin/pandoc`) and already used elsewhere for doc exports.
- Pros: rich markdown fidelity (tables, lists, citations), maintained, battle-tested.
- Cons: a RUNTIME external-process dependency — fragile on hosts/CI without pandoc (must gate on availability with an honest typed error per §11.4.3/§11.4.6, never silent). Lossy round-trip is irrelevant here because the translator's content is already plain markdown-ish text, so pandoc's rich-feature advantage is largely unused. Adds process-spawn + temp-file surface.

(b) Pure-Go OOXML writer (stdlib `archive/zip` + `encoding/xml`)

- The minimal valid WordprocessingML (.docx) part set is small and standardized by ISO/IEC 29500 (verified against the Microsoft Open XML structure doc, see Sources): four parts — `[Content_Types].xml`, `_rels/.rels`, `word/document.xml`, `word/_rels/document.xml.rels` — and the minimal body is `w:document > w:body > w:p > w:r > w:t`.
- Pros: ZERO new dependencies (stdlib only); no runtime external-process failure mode; fully testable in-process; matches the existing pure-Go writer pattern (FB2/EPUB writers spawn no external process); deterministic full control of the

OOXML; non-lossy for the translator's in-memory structured Book (title + chapters/sections → headings + paragraphs).
- Cons: must implement OOXML correctly (mitigated — minimal part set is tiny and standardized; validity asserted programmatically by unzipping + checking the required parts + the translated text, plus a real `file`/`unzip -l` proof and Word-openability).

(c) Maintained Go docx library (unioffice / go-docx v2 / docxgo / gooxml)

- Pros: handles OOXML internals.
- Cons: a new module dependency for a small, well-bounded need; several are heavyweight (unioffice is commercial-licensed for some uses); §11.4.74/ §11.4.28 favour not pulling a dependency we can satisfy with stdlib. Rejected — disproportionate dependency for the scope.

Decision — Option (b), pure-Go OOXML writer

Most robust + lowest-risk for THIS content: the translator output is plain structured text, so pandoc's rich-feature edge is unused while its runtime-process fragility is a real cost; a third-party library is a disproportionate dependency. A pure-Go writer over stdlib `archive/zip` + `encoding/xml`, emitting the ISO/IEC 29500 minimal valid part set, is dependency-free, in-process testable, deterministic, and non-lossy for the Book model. Headings (book title, chapter titles, section titles) are emitted as bold + larger-size paragraphs; body text is split on blank lines into separate `w:p` paragraphs (mirroring the FB2/HTML writers). All text is carried as `w:t` chardata with `xml:space="preserve"` so the `encoding/xml` encoder escapes XML specials automatically (translated content can never inject markup).

Validity is proven programmatically (unzip the produced `.docx`; assert `[Content_Types].xml` + `word/document.xml` present + the translated chapter text appears in `document.xml`) AND on-screen in a real run (`file` reports a Word document, `unzip -l` lists the OOXML parts, the translated text is grep-able in `word/document.xml`). A mutation (break the writer) flips the test RED.

This is NOT a half-baked/lossy writer — for the translator's plain-text book content the OOXML mapping is complete; the §11.4.6 STOP-and-report path (research concludes a quality writer can't be done safely) does NOT apply, because the stdlib path is both safe and quality.

Sources verified

- Microsoft Learn — Structure of a WordprocessingML document (ISO/IEC 29500 minimal document scenario; required `w:document > w:body > w:p > w:r > w:t`; minimal part set): https://learn.microsoft.com/en-us/office/open-xml/word/structure-of-a-wordprocessingml-document (accessed 2026-06-15)
- Pandoc markdown→docx command reference (option (a) baseline): https://opensource.com/article/19/5/convert-markdown-to-word-pandoc (accessed 2026-06-15)

- Go OOXML library survey (option (c) — rejected as disproportionate dependency): https://github.com/unidoc/unioffice , https://github.com/mmonterroca/docxgo (accessed 2026-06-15)

Deep-research 2026-06-15: https://learn.microsoft.com/en-us/office/open-xml/word/structure-of-a-wordprocessingml-document ; https://opensource.com/article/19/5/convert-markdown-to-word-pandoc ; https://github.com/unidoc/unioffice